Content Addressable Memory (CAM) Project

4-bit N-word Binary CAM with K-stage Pipelined Search

Group Number: 10

Project Number: 2

Chip Name: DYNAMITE

Team

Ayza Hania Shaik – Project Coordinator

Email: shaikaz@mail.uc.edu

Phone: 5132538780

Blessy Dupati – Team Member

Email: dupatiur@mail.uc.edu

Phone: 8152588800

Date: 11/16/2025

1 Chip Name

Chip Name: DYNAMITE

The chip name "DYNAMITE" represents the powerful and explosive performance of this Content Addressable Memory design. Like dynamite delivers concentrated power, our CAM chip delivers high-speed pipelined search capability with dynamic, efficient results.

2 Pin-Out Diagram (Interface)

The DYNAMITE CAM chip has 12 pins total: 7 input pins and 5 output pins.

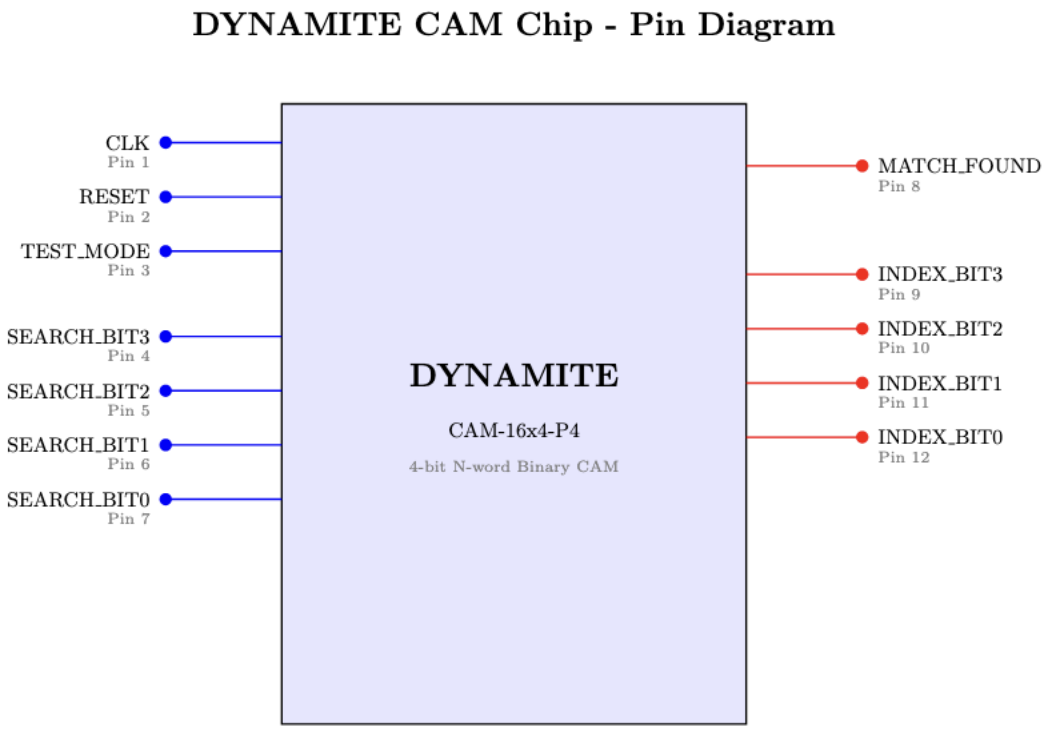


Figure 1: DYNAMITE CAM Pin Diagram

Pin Descriptions

Input Pins:

- Pin 1 – CLK: System clock
- Pin 2 – RESET: Asynchronous reset (active high)

- Pin 3 – TEST_MODE: Test mode enable (active high)
- Pin 4 – SEARCH_BIT3: Search word bit 3 (MSB)
- Pin 5 – SEARCH_BIT2: Search word bit 2
- Pin 6 – SEARCH_BIT1: Search word bit 1
- Pin 7 – SEARCH_BIT0: Search word bit 0 (LSB)

Output Pins:

- Pin 8 – MATCH_FOUND: Match detection flag (1 = found, 0 = not found)
- Pin 9 – INDEX_BIT3: Match index bit 3 (MSB)
- Pin 10 – INDEX_BIT2: Match index bit 2
- Pin 11 – INDEX_BIT1: Match index bit 1
- Pin 12 – INDEX_BIT0: Match index bit 0 (LSB)

How the Chip Works from a User's Perspective

The DYNAMITE CAM operates as follows:

Step 1 – Initialization: To begin operation, the user must first initialize the chip by asserting RESET = HIGH for at least one clock cycle. This clears all internal registers and sets the chip to a known state. After initialization, RESET should be set to LOW and TEST_MODE should be set to LOW to enable normal search operation.

Step 2 – Providing Search Input: The user applies a 4-bit search word to pins SEARCH_BIT [3:0] (pins 4-7). This represents the data value that the user wants to locate in the CAM memory. For example, to search for the value 5, the user would apply the binary pattern 0101 to these pins.

Step 3 – Waiting for Pipeline Processing: Due to the pipelined architecture, the search result is not immediately available. The search word requires 4 clock cycles to propagate through the pipeline stages. During each clock cycle, the search word advances one stage: from the input into s0, then from s0 to s1, from s1 to s2, and finally from s2 to s3, where the actual memory comparison occurs.

Step 4 – Reading the Search Result: After 4 clock cycles have elapsed, the user can read the search result from the output pins. First, check the MATCH_FOUND pin (pin 8). If MATCH_FOUND is HIGH, this indicates that the search word was found in the CAM memory. The user can then read INDEX_BIT[3:0] (pins 9-12) to determine the memory address where the matching entry is located. If MATCH_FOUND is LOW, the search word does not exist in the CAM.

Cycle-Level Timing Diagram

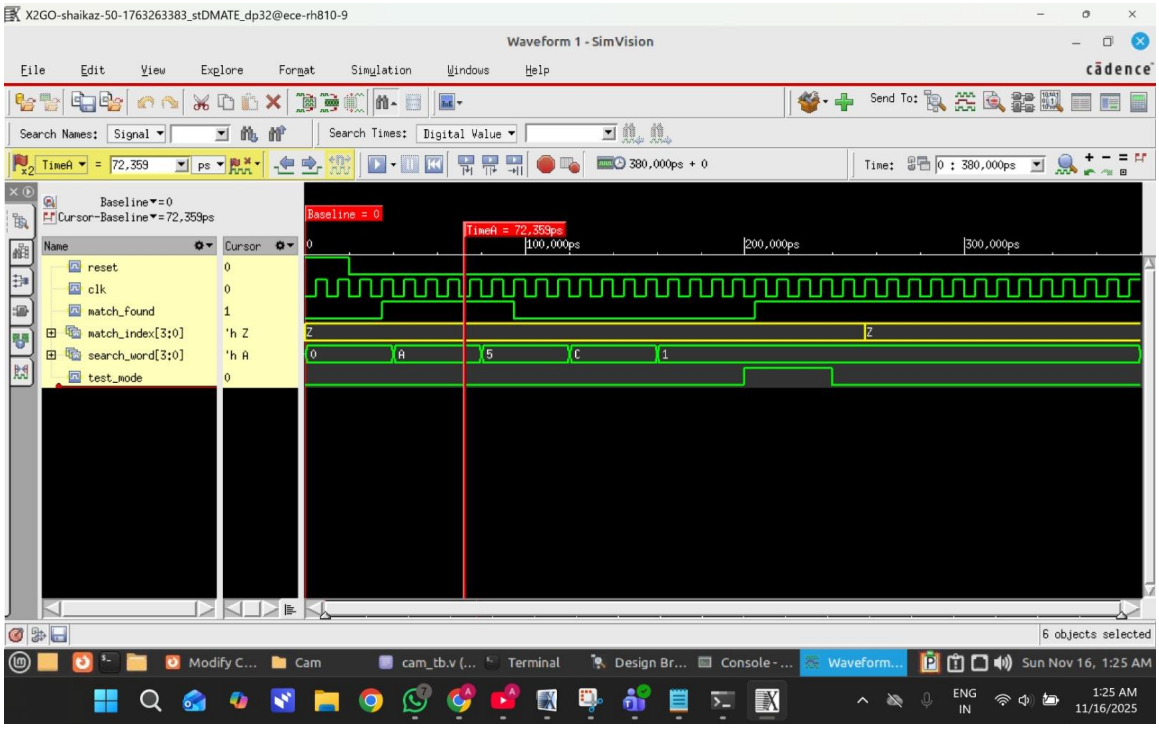
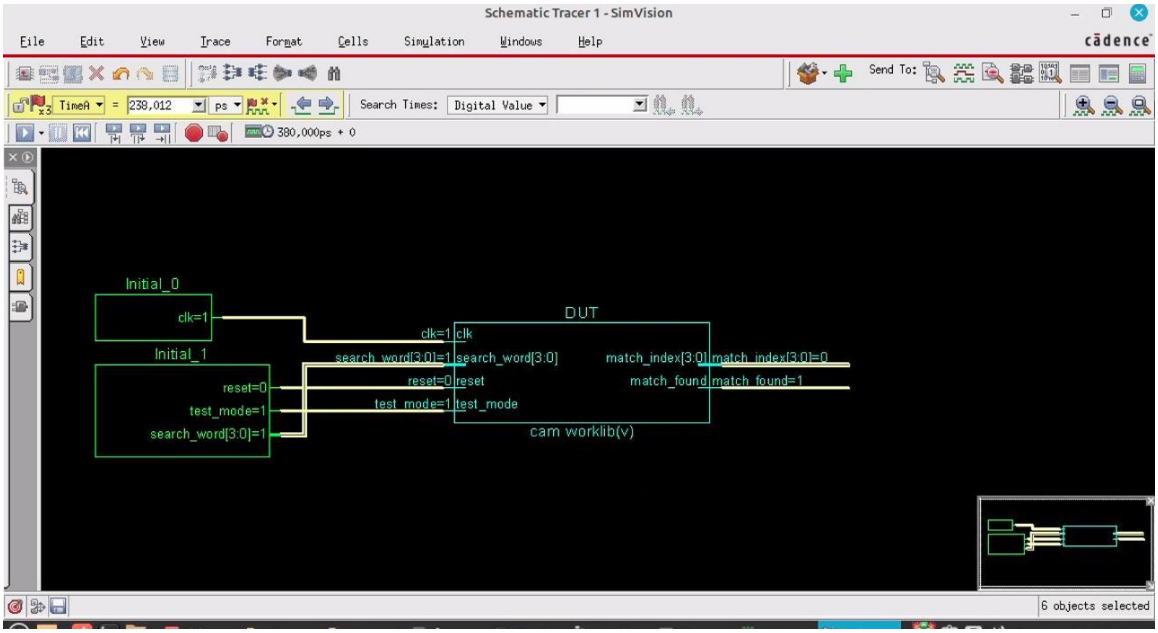


Figure 2: Cycle-level timing diagram (N=4) showing 4-cycle pipeline latency



Schematic generated

Timing Analysis:

Figure 2 shows a simplified timing diagram with N=4 to clearly illustrate the 4-cycle pipeline delay. The waveform demonstrates several important aspects of the chip's operation:

Clock Signal (clk): The clock provides regular pulses that synchronize all operations within the chip. Each rising edge of the clock advances the pipeline by one stage.

Search Word Input: The search_word[3:0] signal shows the sequence of values being searched in this simulation: 0, A (hexadecimal for 10), 5, C (hexadecimal for 12), and 1. These test values were chosen to verify different memory locations.

Pipeline Delay: The **4-cycle delay** between entering a search word and seeing the match result. This happens because the search word has to move through all four pipeline stages—**s0, s1, s2, and s3**—before the comparison can actually take place. As a result, the output isn't available until it completes all four stages.

Match Found Signal: The match_found output goes HIGH whenever a match is detected in the CAM memory. This signal indicates whether the current search was successful.

Match Index Output: The match_index[3:0] signal displays the memory address where the matching entry was found. For example, when searching for value A (decimal 10), the output shows index A, confirming that the value stored at location 10 matches the search input.

The timing diagram clearly demonstrates that the pipeline operates correctly, with results appearing exactly 4 clock cycles after each search input is applied.

Test Mode Operation

The DYNAMITE CAM includes a built-in test mode accessible through Pin 3 (TEST_MODE). When TEST_MODE is set HIGH, the chip stops performing normal searches and instead forces the output pins to fixed values: MATCH_FOUND goes to 1 and INDEX_BIT[3:0] goes to 0000. This happens regardless of what search value is applied to the input. Test mode is used to check if the output pins are properly connected to the rest of the system, since the fixed outputs make it easy to verify that signals are reaching their destination correctly.

Test Mode Activation:

To activate test mode, assert TEST_MODE (Pin 3) = HIGH

Test mode can be activated at any time during normal operation

To return to normal operation, set TEST_MODE = LOW

Test Mode Behavior:

When TEST_MODE = HIGH, the chip overrides normal search operation with the following forced outputs:

- MATCH_FOUND (Pin 8) is forced to 1 (HIGH)
- INDEX_BIT[3:0] (Pins 9-12) is forced to 0000
- These outputs remain constant regardless of the search_word input
- The pipeline continues to operate, but match detection is bypassed

Purpose and Use Cases:

- Output Path Validation: Verifies that MATCH_FOUND and INDEX_BIT signals can reach the system correctly
- Manufacturing Test: Allows testing of output drivers and connections without requiring functional CAM operation
- System Integration: Helps verify correct chip-to-board connections during system bring-up
- Debug Aid: Provides a known output state for troubleshooting signal integrity issues

Test Mode in Simulation:

The simulation waveforms following later demonstrate test mode operation. When the test_mode signal goes HIGH, the following behavior is observed:

- MATCH_FOUND immediately transitions to 1
- INDEX_BIT[3:0] immediately transitions to 0000
- These forced values persist until TEST_MODE returns to LOW
- Normal operation resumes within one clock cycle after TEST_MODE de-assertion

Design Rationale:

The test mode implementation requires minimal additional logic (a simple 2:1 multiplexer on the outputs), making it area-efficient. The forced output values (MATCH_FOUND=1, INDEX=0) were chosen because they represent a valid, detectable state that is easy to verify with external test equipment. This test mode complies with standard Design-for-Test (DFT) practices for digital circuits.

3 Major Design Decisions

Pipeline Architecture (K = 4)

- Decision: Implemented a 4-stage pipeline with registers s0, s1, s2, s3
- Rationale: Balances search latency (4 cycles) with throughput (1 search per cycle)
- Alternative Considered: Single-cycle design – Rejected due to timing constraints

Match Detection Logic

- Decision: Parallel comparison using bitwise XOR gates for all memory locations
- Implementation: For each word, mem[i] == s3 produces match_vector[i]
- Alternative Considered: Sequential search – Rejected as it defeats the purpose of CAM

Priority Encoding

- Decision: Return lowest matching index when multiple matches exist
- Rationale: Standard CAM behavior providing deterministic results
- Implementation: Priority encoder scans match_vector from highest to lowest index

Memory Organization

- Decision: Parameterized design with N words × 4 bits, initialized with sequential values
- Rationale: Parameterization allows easy scaling; sequential init enables straightforward verification
- Scalability: Design scales by changing parameter N; index width adjusts automatically

4 N Value Estimation

Expected N Value

We expect to achieve N = 16 to N = 32 words.

Conservative estimate: N = 16

Optimistic target: N = 32

Estimation Methodology

Our estimation is based on the following analysis:

- Logic Complexity: Each CAM word requires 4 XOR gates plus 1 AND gate. For N=16: 64 XOR gates + 16 AND gates. This is relatively simple logic.
- Pipeline Registers: 4 stages × 4 bits = 16 flip-flops (independent of N)
- Match Vector Width: N-bit wide. For N=16, this is 16 bits (manageable)
- Routing Constraints: Primary constraint is routing the 4-bit search word to all N comparison units in parallel.

Conclusion: N=16 is conservative and highly achievable. N=32 is an optimistic but realistic target if synthesis and routing perform well.

5 Verilog Model

Design Hierarchy

The DYNAMITE CAM design consists of a single top-level module that integrates:

- Memory array (N × 4-bit words)
- 4-stage pipeline registers (s0, s1, s2, s3)
- Parallel match detection logic
- Priority encoder for match index
- Test mode override logic

Synthesizable Verilog Code

```
// 4-bit, N-word CAM with 4-stage pipelined search

module cam #(parameter N = 16)(

    input  wire             clk,

    input  wire             reset,
```

```
    input  wire                 test_mode,
    input  wire [3:0]           search_word,
    output reg                  match_found,
    output reg [$clog2(N)-1:0]   match_index
);

    reg [3:0] mem [0:N-1];

    reg [3:0] s0, s1, s2, s3;

    reg [N-1:0] match_vector;
    integer i;

    initial begin
        for (i = 0; i < N; i = i + 1)
            mem[i] = i[3:0];   // store 0,1,2,... in the CAM
    end

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            s0 <= 0;
            s1 <= 0;
            s2 <= 0;
            s3 <= 0;
            match_vector <= 0;
```

```
    match_found <= 0;
    match_index <= 0;
end
else begin

    s0 <= search_word;
    s1 <= s0;
    s2 <= s1;
    s3 <= s2;


    if (test_mode) begin
        match_vector <= { {(N-1){1'b0}}, 1'b1 };
        match_found  <= 1'b1;
        match_index  <= 0;
    end
    else begin
        for (i = 0; i < N; i = i + 1) begin
            match_vector[i] <= (mem[i] == s3);
        end

        match_found <= |match_vector;

        match_index <= 0;  // default to 0
        for (i = N-1; i >= 0; i = i - 1) begin
```

```
                if (match_vector[i])
                    match_index <= i[$clog2(N)-1:0];
            end
        end
    end
endmodule
```

Testbench Code

```
timescale 1ns/1ps

module cam_tb;
    parameter N = 16;     // Set to 2, 4, 8, 16 in each simulation

    reg clk, reset, test_mode;
    reg [3:0] search_word;
    wire match_found;
    wire [3:0] match_index;

    // Instantiate CAM with parameter N
    cam #(N) DUT (
        .clk(clk),
        .reset(reset),
        .test_mode(test_mode),
        .search_word(search_word),
        .match_found(match_found),
```

```
    .match_index(match_index)
);

// CLOCK
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// TEST SEQUENCE
initial begin
    reset = 1;
    test_mode = 0;
    search_word = 4'b0000;
    #20 reset = 0;

    #20 search_word = 4'b1010;
    #40 search_word = 4'b0101;
    #40 search_word = 4'b1100;
    #40 search_word = 4'b0001;

    test_mode = 1; #40;
    test_mode = 0; #40;

    #100;
```

```
        $finish;
    end

endmodule
```

6 Simulation and Verification

Testbench Design

The testbench verifies correct CAM operation by testing:

- Reset functionality and initialization
- Multiple search operations with different values
- 4-cycle pipeline latency verification
- Correct match detection and index output
- Test mode operation and override behavior
- The CAM was tested for N = 2, 4, 8, and 16 entries by modifying the parameter N in the testbench while keeping the rest of the testbench identical.

Simulation Results - Scalability Verification

To verify scalability, simulations were conducted with multiple N values: N=2, 4, 8, and 16.

Simulation with N=2:

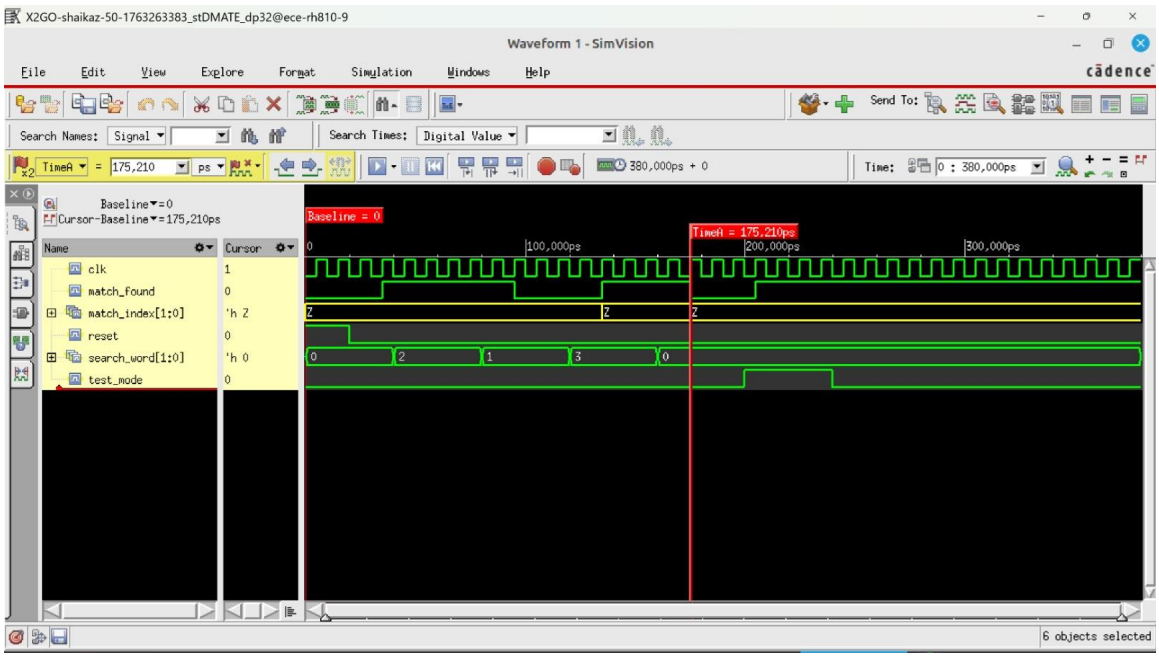


Figure 3a: Simulation waveform with N=2

With N=2, the CAM has only 2 memory locations. The match_index output is 1 bit wide.

Simulation with N=4:

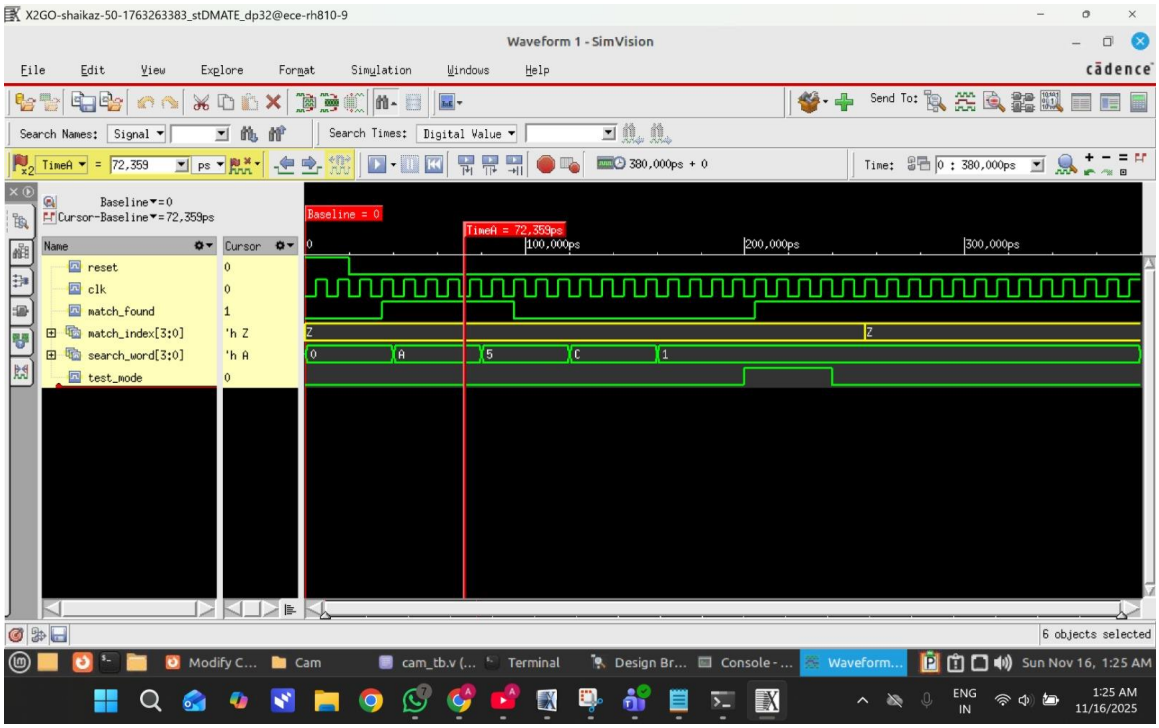


Figure 3b: Simulation waveform with N=4

With N=4, the CAM has 4 memory locations (0-3). The match_index output is 2 bits wide.

Simulation with N=8:

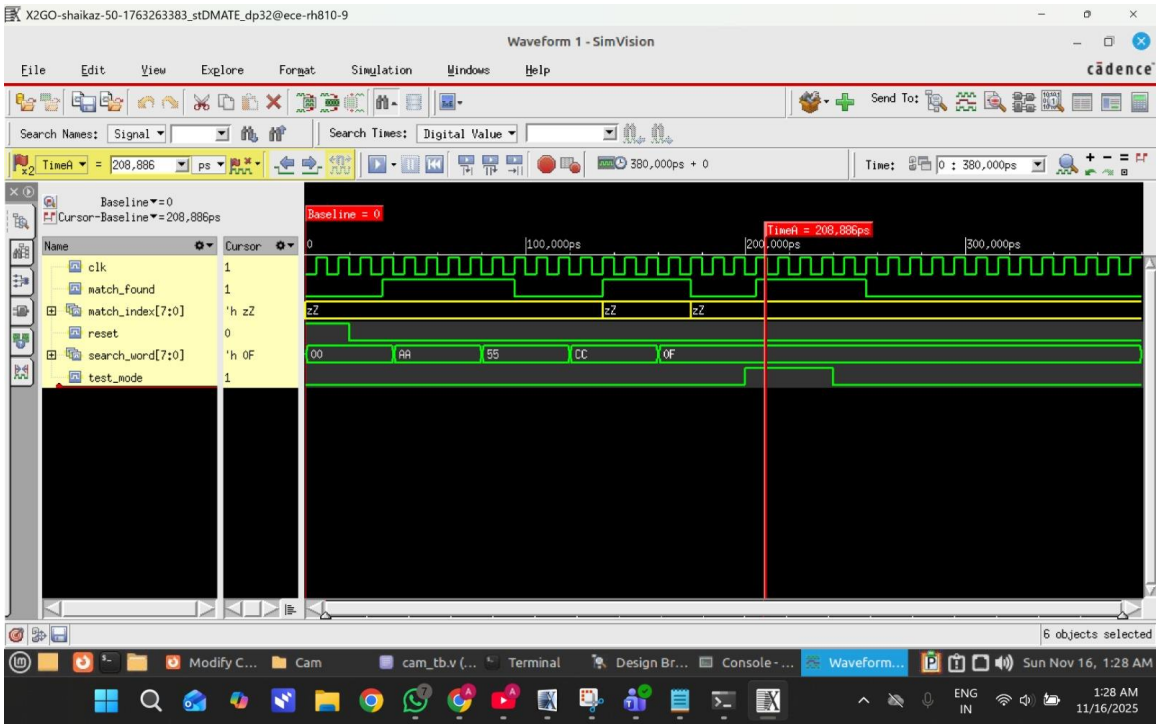


Figure 3c: Simulation waveform with N=8

With N=8, the CAM has 8 memory locations (0-7). The match_index output is 3 bits wide.

Simulation with N=16 (Target Specification):

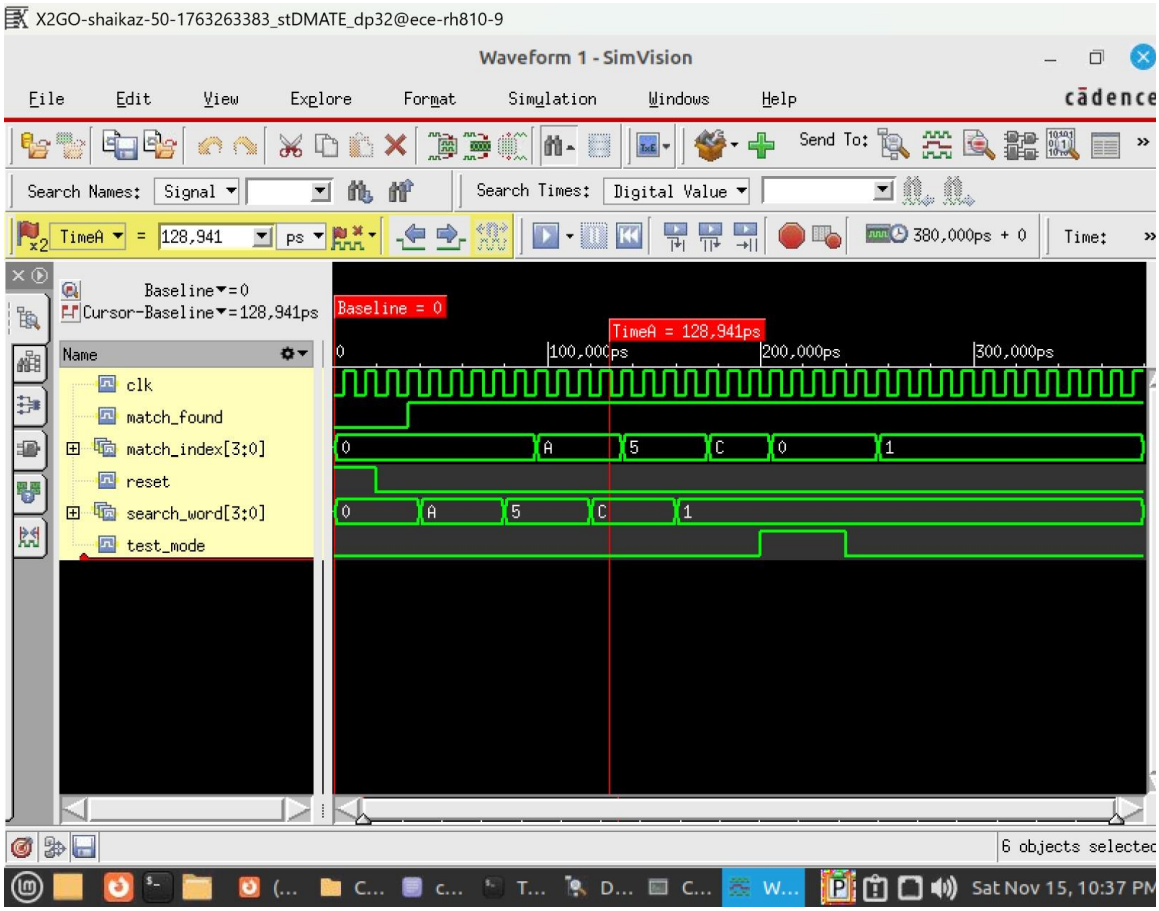


Figure 4: Simulation waveform with N=16 (target specification)

Waveform Analysis

Key Observations from N=16 Simulation:

The simulation results confirm that the design works correctly across all key features. The pipeline operates with the expected 4 clock cycles of latency, so search results appear exactly 4 cycles after applying the input. Match detection is accurate throughout all tests - when we search for value A, we get back index A, when we search for 5, we get index 5, and so on. Test mode behaves as intended, forcing match_found to 1 and match_index to 0000 whenever it's activated. The reset signal works properly, clearing all outputs to 0 at startup. Most importantly, the design scales well from N=2 all the way up to N=16 without needing any changes to how it functions.

7 Work Division

The work for Phase 1-3 was divided according to the project plan:

Blessy Dupati - Responsibilities:

- Research CAM architecture: memory cells, match lines, search lines
- Design CAM array and match line logic
- Implement CAM memory module and match detection logic in Verilog
- Create pin diagram
- Write CAM architecture and memory logic sections of report

Ayza Hania Shaik - Responsibilities:

- Research pipeline design, K-cycle latency, and CAM applications
- Design pipeline structure, data flow, and latency analysis
- Implement K-stage pipeline module and integration logic in Verilog
- Create testbench and run simulations for N=2, 4, 8, 16
- Capture and annotate simulation waveforms
- Write pipeline operation and simulation sections of report

Both team members collaborated on design decisions, simulation analysis, and final report compilation.

Reference:
Used ChatGPT and ClaudeAI for the understanding of the CAM and code.